

```

        System.out.println("Aprobado");
    }
    else
    {
        System.out.println("Suspendido");
    }
}
}

```

Al ejecutar el programa, en la variable *nota* se almacena el valor **8**. A continuación pasa evaluarse la expresión relacional

```

.....
nota >= 5
.....

```

Dado que la variable *nota* almacena **8**, el resultado de la evaluación de la expresión es *true*, y en consecuencia se ejecutan las instrucciones del primer bloque de instrucciones, concretamente sólo la instrucción

```

.....
System.out.println("Aprobado");
.....

```

Cuya acción resultante es que se visualiza en la consola de salida

### Aprobado

Ahora vamos a suponer que para aprobar fuese necesario obtener una nota igual o superior a **5**, y además, haber entregado la tarea encomendada. Con esta ampliación de las condiciones para aprobar, la aplicación quedaría:

```

.....
package utilizacionalternativasimple;

public class UtilizacionAlternativaSimple {

    public static void main(String[] args) {

        byte nota = 8;
        String tareaEntregada = "S";

        if ( nota >= 5 && (tareaEntregada.charAt(0) == 'S' || tareaEntregada.
charAt(0) == 's'))
        {
            System.out.println("Aprobado");
        }
    }
}

```

```

        else
        {
            System.out.println("Suspendido");
        }
    }
}

```

Interviene un nuevo elemento en el programa que es la clase *String*, cuyo estudio en profundidad se realizará en el tema en que se abordarán las Clases envoltorio. Se pretende asociar la circunstancia de haber entregado la tarea con un valor “S” o un valor “s” (en mayúscula o en minúscula) del objeto *String tareaEntregada*. Observemos que lo que realmente utilizamos en el código del programa es ‘S’ y ‘s’. Dichos caracteres van ubicados entre ‘ ‘ porque reciben el tratamiento de *char*, y esta es la forma con que en Java representamos los literales de este tipo de dato. Con lo que la expresión utilizada como condición en la alternativa simple se amplía, y contempla dos expresiones con valor *boolean*:

#### 1. la expresión relacional

```

.....
nota >= 5
.....

```

#### 2. la expresión lógica

```

.....
(tareaEntregada.charAt(0) == 'S' || tareaEntregada.charAt(0) ==
's')
.....

```

Ambas conectadas mediante el operador lógico “&&”, lo cual implica que ambas deben dar un resultado *true* al ser evaluadas para que el valor total de la expresión utilizada como condición sea también *true*. Pero si observamos la segunda, comprobaremos que a su vez es otra expresión lógica que utiliza el operador lógico “||”, lo que implica que para que devuelva *true* al ser evaluada, es suficiente con que devuelva *true* una de las dos expresiones relacionales que actúan como entradas.

Es fácilmente deducible que dada la implementación en código que presenta en estos momentos el programa, siempre se cumplirá la condición y se visualizará **Aprobado** por la consola de salida. Así pues, vamos a dotar de mayores dosis de realismo a nuestro programa haciendo que tanto el valor numérico asociado a la nota, como la respuesta a la pregunta de si se ha entregado la tarea propuesta, sean introducidas por teclado en tiempo de ejecución en sendas peticiones al usuario del programa. Para ello transformaremos nuestro programa en:

---

```

package utilizacionalternativasimple;

import java.io.BufferedReader;
import java.io.InputStreamReader;

public class UtilizacionAlternativaSimple {

    public static void main(String[] args) {

        byte nota = 0;
        String tareaEntregada = "";
        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));
        try {
            System.out.print("Introduzca nota obtenida: ");
            nota = (byte)Integer.parseInt(bufferedReader.readLine());
            System.out.print("Se han entregado las tareas (S/N): ");
            tareaEntregada = bufferedReader.readLine();
        } catch (Exception e)
        {
            System.out.println(e.getMessage());
        }

        if ( nota >= 5 && (tareaEntregada.charAt(0) == 'S' || tareaEntregada.
        charAt(0) == 's'))
        {
            System.out.println("Aprobado");
        }
        else
        {
            System.out.println("Suspendido");
        }
    }
}

```

---

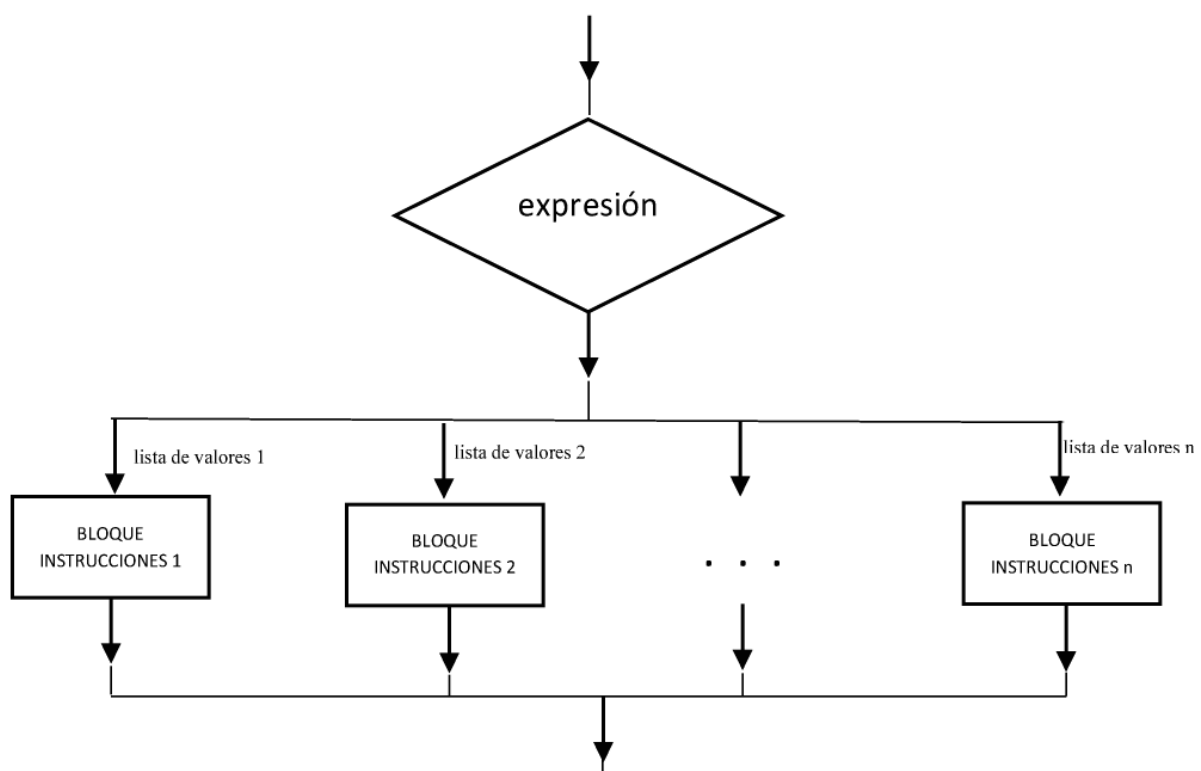
La lógica y propósito del programa sigue siendo los mismos. El cambio aplicado consiste, básicamente, en que en lugar de “fijar” con literales en el código del programa los valores asociados al contenido de la variable *nota*, y del *String tareaEntregada*, sometemos el valor del contenido de ambos a sendas entradas por teclado por parte del usuario en tiempo de ejecución, mediante la invocación al método *readLine()* de la clase *BufferedReader*. La utilización de este método es uno de los mecanismos disponibles en Java para realizar dichas introducciones por teclado en consolas de E/S tipo texto. El estudio de esta clase será abordado con posterioridad en el tema E/S en Java. Al igual que los bloques *try { } catch () { }*

, que se abordará, a su vez en el tema Excepciones: lanzamiento, intercepción y tratamiento. O la invocación del método *Integer.parseInt()*, que devuelve el valor numérico equivalente a una cadena de caracteres, será abordado en el tema en que se tratan las clases envoltorio. Hay que matizar que el valor 0 con que se inicializa la variable *nota* no es significativo como dato en el programa, sino que dicha inicialización es obligatorio realizarla (con cualquier valor) como consecuencia de recoger el valor de *nota* dentro del bloque *try { }*. Lo mismo ocurre con la cadena nula "" con que se inicializa el *String tareaEntregada*.

Por último, en relación con las incorporaciones sintácticas al programa, hay que matizar que se ha aplicado *import*, como consecuencia de haber utilizado las clases *BufferedReader*, e *InputStreamReader*. Estas clases están aglutinadas en el paquete *java.io*. Y en un programa Java es necesario “importar” todas las clases que se vayan a utilizar, no presentes en el paquete en que está la clase actual. Ello aplicable a cualquier clase, bien predefinida, o bien definida por el programador, con excepción de las clases predefinidas presentes en el paquete *java.lang*, el cual es importado por defecto. Como es el caso de las clases *System* y *String* que ya se han venido utilizando en los ejemplos.

## 2.3 ALTERNATIVA MÚLTIPLE

El esquema que le corresponde:



Podemos observar que es similar a la **alternativa simple**, con la diferencia de que la **alternativa múltiple** puede plantear más de 2 posibles caminos alternativos (recordemos, **alternativa simple**: máximo 2 posibles caminos alternativos). El camino alternativo (sólo puede seguir un camino alternativo en cada caso) que tomará el flujo de ejecución del programa será aquel en que alguno de los valores de la **lista de valores** asociada a dicho camino, coincida con el valor resultante de la evaluación en tiempo de ejecución de la **expresión** que acompaña a *switch*. Ahora ya no se admite una expresión *boolean*, dado que un boolean sólo puede tomar dos posibles valores (*true* o *false*). Así pues, la expresión en cuestión deberá ser de uno de los siguientes tipos:

- *byte*
- *short*
- *int*
- *char*

Recientes versiones de Java admiten también como **expresión** que acompaña a *switch*, las correspondientes clases envoltorio a estos tipos primitivos, la clase *String*, y tipos enumerados. Las clases envoltorio, y la clase *String*, se tratarán en el tema correspondiente.

Un ejemplo de codificación en Java de la **alternativa múltiple** es:

```
.....
switch (expresión)
{
    case valor 11:
    case valor 12:
        .
        .
        .
        case valor 1n: BLOQUE DE INSTRUCCIONES 1
                        break;
        case valor 21:
        case valor 22:
        .
        .
        .
        case valor 2n: BLOQUE DE INSTRUCCIONES 2
                        break;
        case valor 31: BLOQUE DE INSTRUCCIONES 3
                        break;
        .
        .
        .
}
```

.....



Las palabras reservadas que se pueden utilizar son:

- ▼ **switch** : Sólo aparece una vez en la estructura, en la cabecera de la instrucción, acompañada por la expresión entre ().
- ▼ **case** : Precede a cada uno de los posibles valores. Cada “pareja” *case valor* supone un potencial “punto de entrada” del flujo de ejecución del programa. Concretamente dicha “entrada del flujo” se producirá en aquella “pareja” tal que el **valor** coincida con el resultado de la evaluación en tiempo de ejecución de la *expresión* que acompaña a *switch*.
- ▼ **break.** : Supone el “punto de salida” del flujo de ejecución desde la estructura. De hecho, en relación al “punto de entrada”, acabamos de describir donde y como se produce. El flujo de ejecución prosigue “descendentemente” (hablamos de la disposición de instrucciones en la estructura). Y la “salida” del flujo de la estructura se produce en el primer *break* con que se encuentra.

Para configurar la estructura en su conjunto, debemos tener en cuenta lo anteriormente descrito en relación a *case* y a *break*; y situar consecutivas tantas “parejas” *case valor* como valores compongan cada lista de valores (remitiéndonos al esquema). A continuación del último *case valor* de cada lista de valores ubicamos todas las instrucciones comprendidas por el bloque de instrucciones asociado a la lista de valores en cuestión. Y a continuación de la última instrucción del bloque, situamos el *break* que determinará el “punto de salida”.

Vamos a ilustrar todo ello con un ejemplo:

```
.....
package utilizacionalternativamultiple;

public class UtilizacionAlternativaMultiple {

    public static void main(String[] args) {
        byte numeroMes = 4;

        switch (numeroMes)
        {
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
```

```

        case 12: System.out.println("el mes tiene 31 dias");
                    break;
        case 4:
        case 6:
        case 9:
        case 11: System.out.println("el mes tiene 30 dias");
                    break;
        case 2: System.out.println("el mes es febrero y puede tener 28 o 29
días segun el año");
                    break;
    }
}
}

```

Analicemos este ejemplo. Cuando se inicia la ejecución del programa, a la variable *numeroMes* se le asigna el valor **4**. A continuación pasa a ejecutarse la **alternativa múltiple**. Ya en la ejecución de la estructura de control, lo primero a que se procede es evaluar la expresión que acompaña a *switch*, en este caso, la expresión está conformada por una sola variable, concretamente *numeroMes*. El resultado de la evaluación de la **expresión**, es, en este caso el valor que almacena dicha variable, concretamente **4**. El siguiente paso (dentro de la ejecución de la **alternativa múltiple**), es buscar si alguno de los valores que acompañan a palabras reservadas *case* coincide con el valor **4**. Lo encuentra, y el *case* asociado determina el “punto de entrada” del flujo de la ejecución del programa. Se procede a ejecutar todas las instrucciones que se encuentra en flujo descendente. Comprobamos que la primera que encuentra y ejecuta es

```

.....
System.out.println("el mes tiene 30 dias");
.....

```

cuya acción resultante es que se visualiza por la consola de salida:

**el mes tiene 30 dias**

Sigue el flujo descendentemente. En este caso, ya no encuentra más instrucciones hasta el *break*; que marca “el punto de salida” del flujo de ejecución del programa. Podríamos decir que dicho flujo “sale” de la **alternativa múltiple**, y pasaría a ejecutar la instrucción que hubiera a continuación.

Este ejemplo serviría para cada uno de los tipos de naturaleza entera, pero no para *char*, dado que nos encontramos con algunos valores (a partir de **10**) que tienen más de un dígito. Podemos ver un ejemplo que sí puede utilizar una **expresión char** :

```

package utilizacionalternativamultiple;

public class UtilizacionAlternativaMultiple {

    public static void main(String[] args) {
        char diaSemana = '4';

        switch (diaSemana)
        {
            case '1': System.out.println("lunes");
                       break;
            case '2': System.out.println("martes");
                       break;
            case '3': System.out.println("miercoles");
                       break;
            case '4': System.out.println("jueves");
                       break;
            case '5': System.out.println("viernes");
                       break;
            case '6': System.out.println("sábado");
                       break;
            case '7': System.out.println("domingo");
                       break;
        }
    }
}

```

Al ejecutar el programa comprobamos que se visualiza en la consola de salida:

**jueves**

En el caso de que el resultado de la evaluación en tiempo de ejecución de la *expresión* que acompaña a *switch* no coincida con ninguno de los valores que acompañen a *case*, es decir, no estuviese contemplado en ninguna lista de valores (remitiéndonos al esquema), el flujo de ejecución del programa no entraría por ningún *case*, es decir, no tomaría ningún camino alternativo. No obstante, la alternativa múltiple admite la utilización de la palabra reservada *default*, lo cual permitiría disponer de un camino alternativo de bloque de instrucciones para el caso que acabamos de plantear. Aplicado al ejemplo anterior:

```

package utilizacionalternativamultiple;

```



```

public class UtilizacionAlternativaMultiple {

    public static void main(String[] args) {

        byte numeroMes = 15;

        switch (numeroMes)
        {
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
            case 12: System.out.println("el mes tiene 31 dias");
                    break;

            case 4:
            case 6:
            case 9:
            case 11: System.out.println("el mes tiene 30 dias");
                    break;

            case 2: System.out.println("el mes es febrero y puede tener 28 o 29
dias segun el año");
                    break;

            default: System.out.println("número de mes incorrecto");
        }
    }
}

```

Al ejecutar el programa, en la variable *numeroMes* se almacena el valor **15**. Obviando los primeros pasos en la ejecución de la **alternativa múltiple**, comprobaremos que se ejecuta el bloque de instrucciones que acompaña a *default*, en este caso, solamente la instrucción

```

System.out.println("número de mes incorrecto");

```

cuya acción resultante es la visualización de

**número de mes incorrecto**

Debemos matizar que la utilización de *default*, aunque viable, es controvertida. Hay algunos círculos académicos en que se considera que proporciona soluciones no estructuradas. Si queremos ceñirnos a esta corriente purista, siempre podremos eludir la utilización de *default* recurriendo a una alternativa que proporciona una solución algorítmica estructurada con el mismo efecto. Pasamos a exponer mediante un ejemplo como podríamos plantear una alternativa estructurada al ejemplo anteriormente expuesto:

```
.....
package utilizacionalternativamultiple;

public class UtilizacionAlternativaMultiple {

    public static void main(String[] args) {

        byte numeroMes = 15;

        if (numeroMes >= 1 && numeroMes <=12)
        {
            switch (numeroMes)
            {
                case 1:
                case 3:
                case 5:
                case 7:
                case 8:
                case 10:
                case 12: System.out.println("el mes tiene 31 dias");
                        break;
                case 4:
                case 6:
                case 9:
                case 11: System.out.println("el mes tiene 30 dias");
                        break;
                case 2: System.out.println("el mes es febrero y puede tener 28 o
29 dias segun el año");
                        break;
            }
        }
        else
        {
            System.out.println("número de mes incorrecto");
        }
    }

}
.....
```

Podemos comprobar que el resultado de la ejecución del programa es totalmente idéntico al anterior. Los cambios operados para prescindir del uso de *default* consisten en disponer a primer nivel de una **alternativa simple**, cuya condición construida de tal manera que devuelve *true* cuando comprueba que el valor resultante de la evaluación de la expresión que acompaña a *switch*, forma parte del conjunto de todos los valores que acompañan a palabras reservadas *case*; en el ejemplo que nos ocupa que el rango de posibles valores de la variable *numeroMes* esté entre **1** y **12**. En este caso se ejecutará el bloque de instrucciones asociado a *true* de la **alternativa simple**, concretamente, se ejecutará la **alternativa múltiple**. En el bloque de instrucciones asociado a *false* ubicamos las mismas instrucciones que en la versión “no estructurada” situaríamos asociadas a *default*.

En este ejemplo se manifiesta por primera vez la posibilidad de incluir una estructura de control, es decir, anidada, como una instrucción más, dentro del bloque de instrucciones de otra, con lo que se consigue una jerarquización por inclusión de unas estructuras en otras. Hay que matizar para entender esto que una estructura de control es una instrucción como otra cualquiera, cuyo único efecto es la alteración en el flujo de ejecución puramente secuencial del programa, como ya se ha explicado con anterioridad.

## 2.4 ESTRUCTURAS REPETITIVAS: WHILE ( ) { }

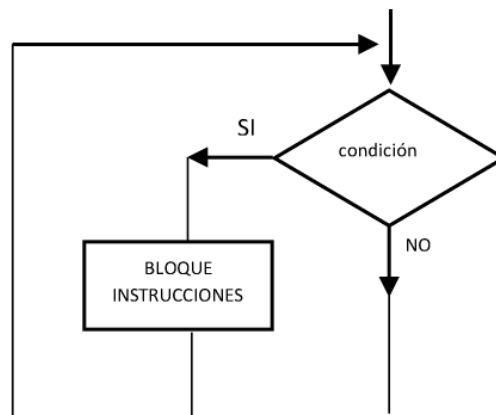
Las estructuras repetitivas suponen una alteración del flujo puramente secuencial de la ejecución del programa sometiendo a repetición a un bloque de instrucciones. El número de repeticiones que vayan a tener lugar, dependerá siempre de que una determinada condición se cumpla. Cuando dicha condición deje de cumplirse, ya no tendrán lugar más repeticiones, y se dará por finalizada la ejecución de dicha estructura repetitiva. El modo y lugar en que planteemos la condición, dará lugar a las diferentes variantes de estructura repetitiva. Una primera variante, cuyo tratamiento abordaremos en este apartado es:

```

.....
while (expresión que al ser evaluado devuelve un valor boolean)
{
    instrucción 11;
    instrucción 12;
    ...
}
.....

```

que responde al siguiente esquema:



Observando el esquema deducimos que el punto de entrada a la estructura de control, supone evaluar la condición. Dicha condición es una expresión *boolean*, que ubicamos entre ( ), y que responde, exactamente, a las características descritas anteriormente, a aplicar a la estructura de control alternativa simple. Si la expresión *boolean*, al ser evaluada, devuelve *true*, el flujo de ejecución toma el camino marcado con **SI** en el esquema, y el camino marcado con **NO**, en caso contrario. El camino determinado por el SI supone la ejecución del bloque de instrucciones, y vuelta a evaluar la condición, que, si sigue cumpliéndose, vuelve a repetirse el bloque de instrucciones. En dicho bloque de instrucciones debe haber, necesariamente, alguna instrucción tal que, al ejecutarse, haga que la condición deje de cumplirse, de tal modo que, en la siguiente evaluación de la condición, al no cumplirse, es decir, al devolver *false*, el flujo de ejecución tomará el camino marcado por el **NO**, lo que supondrá el fin de la ejecución de la estructura repetitiva.

Ilustremos el funcionamiento de esta estructura repetitiva con diversos ejemplos.

```

package utilizacionwhile;

public class UtilizacionWhile {

    public static void main(String[] args) {
        int contador = 1;
        while ( contador <= 10 )
            contador++;

        System.out.println("al finalizar la repetición la variable contador tiene
el valor de : "+contador);
    }
}
  
```

La ejecución del ejemplo visualiza en la consola de salida:

**al finalizar la repetición la variable contador tiene el valor de : 11**

Encontramos una discrepancia con el formato sintáctico expuesto, en relación a “ {} ” que actúan como delimitadores. Se han omitido en este ejemplo, porque su uso es opcional cuando el bloque de instrucciones está constituido por una sola instrucción, análogamente a lo que ocurre con la alternativa simple. Declaramos la variable *int contador*, inicializándola a 1. Al “entrar” el flujo de ejecución en la estructura repetitiva, la primera actuación es evaluar la condición. Con el primer valor que toma la variable *contador*, la expresión devuelve *true* al ser evaluada, con lo que ejecuta el bloque de instrucciones. En este caso, el bloque de instrucciones está constituido por una sola instrucción: el incremento de la variable. Acto seguido, vuelve a evaluar la condición, que al cumplirse, vuelve a repetirse el bloque de instrucciones, volviendo a incrementar la variable. Y así sucesivamente, llega un momento en que la variable *contador* almacena valor 11, que aplicado a la expresión relacional que conforma la condición, hace que devuelva *false*, llegando a su fin la ejecución de la estructura repetitiva. Es muy importante detectar que hay una acción en el bloque de instrucciones, en este caso la única, que hace que en un momento determinado deje de cumplirse la condición, y finalice la repetición.

Aplicando una modificación al ejemplo:

```
.....
package utilizacionwhile;

public class UtilizacionWhile {

    public static void main(String[] args) {
        int contador = 1;
        while ( contador <= 10 ) {
            System.out.println("el valor de la variable contador es : "+conta-
dor);
            contador++;
        }
    }
}
.....
```

al ejecutarse se visualiza en la consola de salida:

- el valor de la variable contador es : 1
- el valor de la variable contador es : 2
- el valor de la variable contador es : 3



- el valor de la variable contador es : 4
- el valor de la variable contador es : 5
- el valor de la variable contador es : 6
- el valor de la variable contador es : 7
- el valor de la variable contador es : 8
- el valor de la variable contador es : 9
- el valor de la variable contador es : 10

Ahora, nos hemos visto obligados a delimitar mediante “`{}`” el bloque de instrucciones, motivados porque está constituido por más de una instrucción.

Planteamos a continuación otro ejemplo, con la misma estructura repetitiva, pero contando, y acumulando el valor de los números pares contemplados en el intervalo de la variable de control aplicada a la condición. A los números impares de dicho intervalo se les aplica un tratamiento análogo:

```
.....
package utilizacionwhile;

public class UtilizacionWhile {

    public static void main(String[] args) {
        int contadorPares = 0;
        int acumuladorPares = 0;
        int contadorImpares = 0;
        int acumuladorImpares = 0;

        int i = 1;
        while ( i <= 10 ) {
            if (i % 2 == 0)
            {
                contadorPares++;
                acumuladorPares+=i;
            }
            else
            {
                contadorImpares++;
                acumuladorImpares+=i;
            }

            i++;
        }
        System.out.println("Se han procesado "+contadorPares+" números pares, y
su suma es "+acumuladorPares);
    }
}
```

```

        System.out.println("Se han procesado "+contadorImpares+" números impa-
res, y su suma es "+acumuladorImpares);
    }
}

```

---

La ejecución de este programa ejemplo visualiza en la consola de salida:

- ▀ Se han procesado 5 números pares, y su suma es 30
- ▀ Se han procesado 5 números impares, y su suma es 25

La intención didáctica de este último ejemplo es mostrar el anidamiento de una estructura de control (alternativa simple), como una instrucción más del bloque de instrucciones de la estructura repetitiva.

Los ejemplos presentados aplicados a la estructura repetitiva *while* suponen la utilización de dicha estructura repetitiva para la implementación de un “mecanismo contador”. Dicho mecanismo, es un proceso que nos permite disponer de una variable de control que, partiendo de un valor inicial, se incrementa progresivamente hasta llegar a un valor final, y para cada uno de los valores de la mencionada variable de control, se aplica una repetición del bloque de instrucciones. Exponemos a continuación, otro ejemplo en que se utiliza la estructura repetitiva *while*:

---

```

package divisionrestassucesivas;

import java.io.BufferedReader;
import java.io.InputStreamReader;

public class DivisionRestasSucesivas {

    public static void main(String[] args) {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));
        int dividendo = 0, divisor = 1, cociente, resto;

        try {
            System.out.print("Introducir dividendo : ");
            dividendo = Integer.parseInt(bufferedReader.readLine());

            System.out.print("Introducir divisor : ");
            divisor = Integer.parseInt(bufferedReader.readLine());
        } catch (Exception excepcion)
        { System.out.println(excepcion.getMessage());

```

```

    }

    cociente = 0;
    while (dividendo >= divisor)
    {
        dividendo -= divisor;
        cociente++;
    }
    resto = dividendo;
    System.out.println("El cociente es : "+cociente+"    y el resto : "+res-
to);
    }
}

```

La ejecución del programa visualiza en la consola de salida:

- Introducir dividendo : 28
- Introducir divisor : 3
- El cociente es : 9 y el resto : 1

Se trata, como vemos, de implementar un algoritmo de división entera mediante restas sucesivas, en que dados dos valores solicitados por teclado, correspondientes a dividendo y divisor, obtenemos el cociente y el resto. Obviamente, el interés del algoritmo es meramente didáctico, dado que para ello ya disponemos en Java de los operadores “/” y “%”. Observamos que la condición para la repetición:

```

while (dividendo >= divisor)

```

no va enfocada al control de un “mecanismo contador” como el descrito, en que la variable de control que actúa como contador, “marcaba el ritmo” de las repeticiones. Ahora, la variable que se incrementa con cada repetición (cociente), actúa como contador “registrador de sucesos”. En el caso de este ejemplo, el suceso a registrar (o a contar), es el número de veces que al dividendo le podemos restar el divisor, es decir el cociente.

## 2.5 ESTRUCTURAS REPETITIVAS: DO { } WHILE ( )

Java dispone de otra estructura repetitiva, con mínimas diferencias respecto a la anterior, cuyo formato sintáctico es:

---

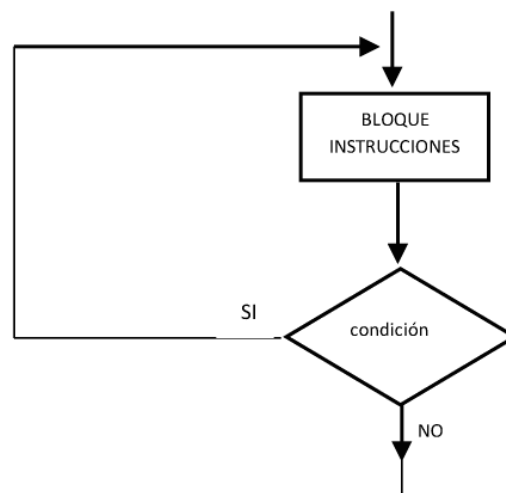
```

do {
    instrucción 11;
    instrucción 12;
    ...
} while (expresión que al ser evaluado devuelve un valor boolean);

```

---

A la que corresponde el esquema:



La diferencia respecto a la estructura repetitiva *while { }* radica en que la condición se evalúa después de la ejecución del bloque de instrucciones. Lo cual trae como consecuencia que, el bloque de instrucciones se ejecuta, al menos, una vez. En el caso de la repetición *while { }*, si de partida la condición no se cumple, el bloque de instrucciones no se ejecuta ni una sola vez. El criterio para implementar la estructura repetitiva *do { } while*, en lugar de la anterior, es que las acciones que deben tener lugar para poder aplicar la condición, formen parte del bloque de instrucciones.

Exponemos un ejemplo en que se implementa esta estructura repetitiva:

---

```

package utilizaciondowhile;

import java.io.BufferedReader;
import java.io.InputStreamReader;

public class UtilizacionDoWhile {

    public static void main(String[] args) {

        BufferedReader bufferedReader = new BufferedReader(new

```

```

InputStreamReader(System.in));
    String cadenaRepeticion , cadenaAConvertir;
    int numero;
    int contador = 0;
    int acumulador = 0;

    try {
        do {
            System.out.print("Introducir número : ");
            cadenaAConvertir = bufferedReader.readLine();
            numero = Integer.parseInt(cadenaAConvertir);
            contador++;
            acumulador+=numero;
            System.out.print("Introducir otro número (S/N) : ");
            cadenaRepeticion = bufferedReader.readLine();
        } while ((cadenaRepeticion.charAt(0) == 'S' || cadenaRepeticion.
charAt(0) == 's') && ((contador+1) <= 5));

        System.out.println("Se han introducido un total de "+contador+"
números");
        System.out.println("El valor acumulado de todos ellos es : "+acumu-
lador);
    } catch (Exception excepcion)
    { System.out.println(excepcion.getMessage());
    }
}
}

```

La ejecución del ejemplo aporta la siguiente visualización a la consola de salida:

```

Introducir número : 12
Introducir otro número (S/N) : S
Introducir número : 7
Introducir otro número (S/N) : S
Introducir número : 11
Introducir otro número (S/N) : N
Se han introducido un total de 3 números
El valor acumulado de todos ellos es : 30

```

El tratamiento aplicado en la repetición es contar, y acumular el valor de los números introducidos por teclado. Para dicha introducción por teclado, se han utilizado mecanismos ya comentados con anterioridad. En cada repetición, después



de dar el citado tratamiento al valor numérico introducido, se le pregunta al usuario del programa si desea introducir otro número. Respuesta afirmativa a dicha pregunta es una de las exigencias para proseguir con la repetición, implementada mediante la expresión:

```
(cadenaRepeticion.charAt(0) == 'S' || cadenaRepeticion.charAt(0) == 's')
```

Como máximo se admiten 5 números, exigencia implementada por:

```
((contador+1) <= 5)
```

La condición global supone que se cumplan ambas exigencias, por ello se conforma conectando ambas expresiones mediante el operador &&.

Exponemos a continuación otro ejemplo:

```
package ejemplomenu;

import java.io.BufferedReader;
import java.io.InputStreamReader;

public class EjemploMenu {

    public static void main(String[] args) {
        BufferedReader bufferedReader = new BufferedReader(new
        InputStreamReader(System.in));
        String opcion;

        try {
            do {
                System.out.println("----- MENU -----");
                System.out.println("1.- Mantenimiento clientes");
                System.out.println("2.- Introducción de facturas");
                System.out.println("3.- Listado de facturas");
                System.out.println("4.- Finalizar");
                System.out.print("Introduzca número opción escogida : ");
                opcion = bufferedReader.readLine();

                switch (opcion)
                {
                    case "1": System.out.println("Ejecutaría mantenimiento de
```

```

clientes");
                break;
            case "2": System.out.println("Ejecutaría introducción de
facturas");
                break;
            case "3": System.out.println("Ejecutaría listado de factu-
ras");
                break;
            case "4": System.out.println("Ha optado por cerrar la
aplicación");
                System.exit(0);
                break;
        }

        } while (opcion.compareTo("4") != 0);
    } catch (Exception excepcion)
    { System.out.println(excepcion.getMessage());
    }
}
}

```

Al ejecutar el programa se visualiza en la consola de salida:

```

----- MENU -----
1.- Mantenimiento clientes
2.- Introducción de facturas
3.- Listado de facturas
4.- Finalizar
Introduzca número opción escogida : 1
Ejecutaría mantenimiento de clientes
----- MENU -----
1.- Mantenimiento clientes
2.- Introducción de facturas
3.- Listado de facturas
4.- Finalizar
Introduzca número opción escogida : 3
Ejecutaría listado de facturas
----- MENU -----
1.- Mantenimiento clientes
2.- Introducción de facturas
3.- Listado de facturas
4.- Finalizar
Introduzca número opción escogida : 4

```